Random Forest Optimisation for Multiclass Cyberattack Prediction

Student Number: 28621883

***Abstract*—Cyberattack classifiers must be able to recognise rare cyber attacks without allowing common traffic classes to dominate the model's selection choices. This study investigates a random forest ensemble on an imbalanced ten-class network-traffic dataset. The ensemble model extends the stronger of two earlier baselines: a single classification tree and a k-nearest neighbours classifier. A fixed, group-aware test partition enables direct comparison with both baselines. Three-fold stratified, group-aware cross-validation was used for experiments testing ensemble size, feature subsampling, tree complexity and class weighting. The selection procedure prioritised macro-F1 and used minority-class recall to distinguish configurations that produce statistically similar macro-F1 values. The selected forest contains 200 entropy-based trees. Each split considers half of the transformed predictors. Trees have an unrestricted depth, a minimum leaf size of five and balanced class weights. The ensemble achieved cross-validation macro-F1 of 0.6147 ± 0.0058 and test macro-F1 of 0.6357. The corresponding test macro-F1 scores were 0.4751 for k-nearest neighbours and 0.5899 for the classification tree. The ensemble increased recall substantially for several rare classes. Test accuracy decreased by 0.0047 relative to the classification tree. The results show that a tuned, cost-sensitive forest produced more balanced cyberattack predictions, although one attack class remained difficult to identify.**

I. Introduction

Network attack detection requires discernment between normal traffic and multiple attack categories. The class distribution often contains many common observations and few examples of actual rare attacks. A classifier optimised for overall accuracy can therefore perform well while missing security-relevant minority classes [5]. Model evaluation therefore should consider both the overall correctness and performance across individual classes.

An earlier study (assignment 2) applied k-nearest neighbours (KNN) and a classification tree to an edited University of New South Wales Network-Based 2015 (UNSW-NB15) dataset. The classification tree achieved higher performance than KNN. A single tree nevertheless has high variance because small changes in the training data can alter early splits [6]. Ensemble learning provides a direct way of determining whether aggregation reduces that limitation.

The primary research question asks whether a tuned random forest improves multiclass cyberattack classification, particularly macro-F1 and minority-class recall, relative to the KNN and classification-tree baselines. Four supporting questions guide optimisation (and the choice of hyperparameter tuning). The questions concern the effects of ensemble size, feature subsampling, tree complexity and class weighting. Aggregation is expected to reduce the variance of a single tree. Balanced weighting is expected to improve rare-class recall at a possible cost to accuracy.

The study retains the data partition and evaluation measures from the earlier comparison in assignment 2. The use of a common design across the models means that performance changes can be primarily attributed to ensembling and cost-sensitive training. The report first presents the required background. Implementation and empirical procedures follow. Results then address every research question before the final model comparison and conclusion.

II. Background

This section introduces the prediction problem and the three learning approaches compared in the study. Ensemble diversity, random forest construction, class imbalance and evaluation measures provide the theoretical basis for the empirical design choices.

A. Network-Traffic Classification

The original UNSW-NB15 dataset combined modern normal traffic with synthetic contemporary attack activity [7]. Each observation describes a network event using packet, byte, timing, state, service, protocol, load and connection-count features. The edited assignment dataset contains 257,673 observations, 43 descriptive features, and ten target classes. Class 0 contains 93,000 observations, whereas class 7 contains only 174 observations. Such an imbalance complicates both learning and evaluation.

The assignment data retain several quality issues. The categorical `service` feature contains 141,321 missing values. Numerical predictors have different ranges and positive outliers. Several predictors are strongly correlated. The `id` feature uniquely identifies observations rather than network behaviour. After removal of `id`, 94,928 repeated labelled observations remain. There are also 2,445 identical predictor patterns associated with conflicting labels. Dirty data can affect learning algorithms differently [9]. Preprocessing must therefore reflect the assumptions of each model.

B. Nearest-Neighbour Classification

KNN is a non-parametric classifier that stores labelled training observations. A new observation receives a label from the classes of nearby observations [2]. The number of neighbours controls local flexibility. Small neighbourhoods follow fine-grained structure but are sensitive to noise. Larger neighbourhoods smooth decision boundaries.

Distance calculations make KNN sensitive to numeric scale, outliers, redundant dimensions and categorical encoding [3]. The Assignment 2 implementation used standardisation and one-hot encoding. Three near-redundant predictors were removed. The selected model used five neighbours, Manhattan distance and distance-weighted voting. KNN provides a structurally dissimilar baseline (when compared with the trees) because prediction depends on local distance rather than learned tree partitions.

C. Classification Trees

A classification tree recursively partitions the predictor space into increasingly homogeneous regions [6]. For C classes with node proportions $p_1, \ldots, p_C$, entropy is

$$H = -\sum_{c=1}^{C} p_c \log_2(p_c). \tag{1}$$

The tree selects splits that reduce impurity. To assign a class value the prediction follows the learned branch conditions to a terminal node.

Monotonic scaling does not alter the order of numeric values or the available threshold partitions. Trees therefore do not require standardisation. Outliers affect trees less directly than distance-based classifiers, although the deep branches can fit small regions. Maximum depth and minimum leaf size regulate model complexity. The selected Assignment 2 tree used entropy, maximum depth 20, minimum leaf size 10 and no class weighting.

D. Ensemble Learning

An ensemble combines predictions from multiple fitted models. Diversity is essential because identical models repeat the same errors. Accurate and diverse members allow aggregation to cancel uncorrelated errors [4].

Bagging fits each base learner on a bootstrap sample of the training data. Classification combines the resulting predictions by voting. Let B denote the number of learners and let $h_b(\boldsymbol{x})$ denote the class predicted by learner b for feature vector $\boldsymbol{x}$. The ensemble prediction is

$$\hat{y}(\boldsymbol{x}) = \underset{c \in \{1,\ldots,C\}}{\arg\max} \sum_{b=1}^{B} \mathbb{I}\left[h_b(\boldsymbol{x}) = c\right]. \tag{2}$$

Bootstrap variation reduces variance most effectively when the individual learners do not make strongly correlated errors.

E. Random Forests

A random forest (RF) combines bootstrap aggregation with random feature selection [1]. Each tree receives a bootstrap sample. Each split considers only a randomly selected subset of predictors. Feature restriction decorrelates trees because strong predictors cannot dominate every split. Excessive restriction weakens individual trees. Insufficient restriction produces similar trees. Generalisation therefore depends on both member strength and member correlation [1].

The number of trees mainly controls the stability of the aggregate prediction. Performance generally approaches a plateau as the forest grows. Maximum depth and minimum leaf size control individual-tree complexity. Feature subsampling and bootstrap sampling control diversity.

Balanced class weighting changes the influence of each class during tree induction. For n training observations, C classes and n_c observations from class c, balanced weight is

$$w_c = \frac{n}{C n_c}. \tag{3}$$

Rare classes therefore contribute more strongly to impurity reduction. Weighting can increase rare-class recall while decreasing majority-class accuracy. Such trade-offs are expected in imbalanced learning [5].

F. Performance Measures

Accuracy measures the proportion of correct predictions. Accuracy alone, however, can hide weak minority class performance. Precision measures the proportion of predictions for a class that are correct. Recall measures the proportion of a class that is identified. F1 is the harmonic mean of precision and recall. Macro-F1 averages class-specific F1 values without frequency weighting and therefore gives every class equal influence [11].

Cohen's kappa measures agreement beyond chance. The Matthews correlation coefficient (MCC) summarises association using the complete multiclass confusion matrix. No single measure describes every error characteristic [10]. The study therefore uses macro-F1 for model selection and reports accuracy, kappa, MCC, per-class precision, recall, F1 and confusion matrices as supplementary information. The background establishes the need for an ensemble that balances diversity, member strength and class sensitivity.

III. Implementation

This section describes the software pipeline and the model-specific transformations. The implementation used Python with Pandas and NumPy for data handling. Scikit-learn supplied preprocessing, partitioning, models and metrics. Matplotlib and Seaborn produced figures. Joblib stored the fitted pipeline. The source code and dependency list are available in the project repository.[1]

A. Common Data Preparation

The comma-delimited data were read with "?" interpreted as missing. No target labels were missing. The `id` predictor was removed because row identity does not describe network behaviour. Repeated observations were retained because repeated network events are not necessarily erroneous. Deletion would also alter the class distribution.

A hash of all untransformed predictors defined a group identifier. Equal predictor patterns remained on the same side of every partition. Grouping prevented a zero-distance duplicate from appearing across training and evaluation data. Preprocessing was contained inside every model pipeline. Imputation and encoding were therefore learned only from the applicable training fold.

[1] https://github.com/jacohuman/machine-learning-assignment4

B. Model Preprocessing

The KNN baseline used median numeric imputation, standardisation, missing-category replacement and one-hot encoding. Categorical levels occurring fewer than 20 times were pooled. Three redundant predictors were removed to prevent repeated distance weighting.

The tree and forest used median numeric imputation. Missing categorical values formed a dedicated `Missing` category. The `proto`, `state` and `service` predictors were one-hot encoded. Levels occurring fewer than 20 times were pooled. Numeric scaling, outlier transformation and correlation-based removal were omitted. Those operations do not change the order-based split candidates and would reduce comparability with the Assignment 2 tree.

C. Random Forest Classifier

The forest used entropy-based trees and bootstrap sampling. Every candidate configuration used a fixed random seed of 42. Parallel tree construction reduced fitting time. In the end the frozen model contained 200 trees. Each split considered 50% of the transformed predictors. Trees had unrestricted depth and a minimum leaf size of five. The use of balanced class weights meant that the influence of rare attack categories was increased. The complete preprocessing and model pipeline was fitted on 206,254 training observations. The fitted pipeline was then stored for reproducibility.

IV. Empirical Process

This section defines the model-selection questions, partitions, measures and staged procedure. Every parameter tuning decision was done incrementally and was done before the final test evaluation. The common held out partition was evaluated once after the configuration was frozen.

A. Research Questions

The main question asked whether a tuned forest improved multiclass cyberattack classification relative to KNN and a single classification tree. Macro-F1 and minority-class recall defined improvement. Four subordinate questions were investigated:

1) How many trees are required before performance reaches a plateau?
2) How does feature subsampling change diversity, performance and fitting cost?
3) How does depth and leaf-size restrictions change underfitting and overfitting?
4) Does class weighting improve minority recall without leading to too much loss in macro-F1?

B. Data Partitioning

The exact Assignment 2 outer split was reused. The training partition contained 206,254 observations. The untouched test partition contained 51,419 observations. Saved row indices reproduced both partitions. Validation confirmed zero overlapping indices, zero unassigned observations and zero predictor groups shared across the boundary.

Three-fold stratified, group-aware cross-validation (CV) operated inside the training partition. Stratification approximated the target distribution in every fold. Grouping prevented equal predictor patterns from crossing training and validation boundaries. Three folds matched the baseline experiments (of assignment 2) and kept repeated forest fitting feasible.

C. Selection Criteria

Mean CV macro-F1 was the primary criterion. Fold standard deviation described variability. Accuracy and fitting time supported the interpretation. Mean recall across classes 2, 5, 7, and 8 provided an additional measure during class-weight and confirmation experiments since these classes exposed the clearest weaknesses on the baselines.

A candidate qualified when mean macro-F1 was no more than one standard error below the highest mean in order to avoid unnecessary model complexity. The smallest qualifying forest was preferred during ensemble-size tuning. During final confirmation, the qualifying configuration with the highest minority recall was selected.

D. Staged Optimisation

Tuning was done in incremental stages. The first experiment compared 50, 100, 200 and 400 trees. The second experiment compared `log2`, `sqrt`, 0.25, 0.5 and 1.0 feature fractions. Leaf sizes of 1, 5, 10 and 25 were then compared at depth 20. Maximum depths of 10, 20, 30 and unrestricted depth were compared after leaf-size selection. The class-weight experiment compared no weighting, globally balanced weights and weights recalculated for each bootstrap sample.

These sequesntial one-parameter experiments can miss interactions, therefore a focused check was done afterwards to compare the sequentially selected model with three alternatives. The alternatives used all predictors, depth 30 or leaf size five. The selected model was then fitted to the complete training partition. The final test prediction produced every reported test measure, confusion matrix and class-specific result from one evaluation.

V. Results and Discussion

A. Ensemble Size and Feature Subsampling

Table I reports the forest-size experiment. Validation macro-F1 reached its highest mean at 200 trees. Increasing the forest to 400 trees doubled mean fitting time from 7.5 to 15.7 seconds per fold and reduced mean macro-F1 slightly. Two hundred trees was therefore selected. Figure 1 shows the performance plateau and approximately linear fitting-cost growth.

TABLE I
Ensemble-size cross-validation results

Trees	Train F1	Validation F1	Time (s)
50	0.4932	0.4619 ± 0.0016	2.2
100	0.4936	0.4614 ± 0.0030	3.9
200	0.4945	0.4637 ± 0.0026	7.5
400	0.4945	0.4626 ± 0.0019	15.7

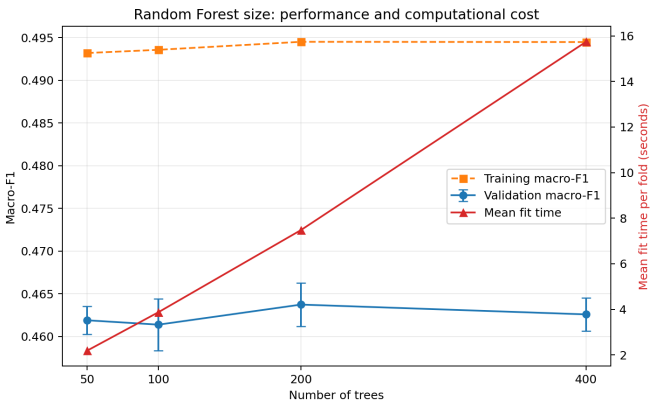


Fig. 1. Predictive performance and cost for different ensemble sizes.

The initial square-root feature rule produced macro-F1 of 0.4637. Figure 2 shows a large improvement as more features became available. A feature fraction of 0.5 achieved 0.5762 ± 0.0044. Using every feature achieved 0.5785 ± 0.0065. The full feature configuration required 82.0 seconds per fold, compared with 39.5 seconds for the 0.5 fraction. The 0.5 fraction also preserved feature-based diversity and produced higher accuracy. Half of the transformed predictors were therefore selected.

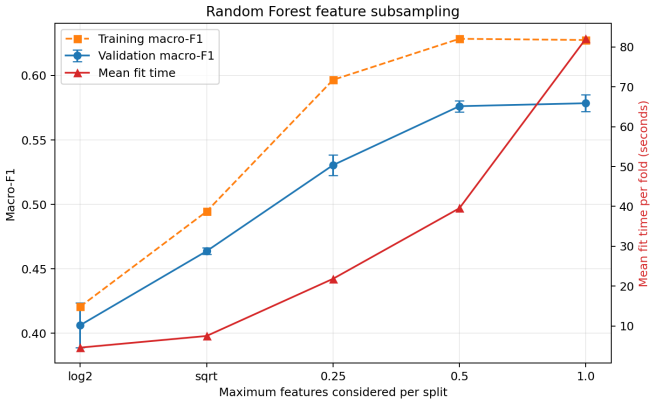


Fig. 2. Effect of feature subsampling on validation performance and cost.

The feature experiment showed that excessive randomisation weakened the individual trees. The result supports the strength-correlation trade-off described for random forests [1].

B. Tree Complexity and Class Weighting

Reducing minimum leaf size from 10 to one increased validation macro-F1 from 0.5762 to 0.5849. Training macro-F1 increased from 0.6285 to 0.7392. The larger gap indicated greater member complexity, but validation performance still improved. Depth 10 underfit with validation macro-F1 of 0.5046. Unrestricted depth achieved the highest mean of 0.5906. Depth 30 achieved 0.5882 and provided almost no fitting-time saving. Unrestricted depth was retained before class weighting.

Balanced weights raised mean CV macro-F1 from 0.5906 to 0.6162. Mean minority recall rose from 0.3301 to 0.4061. Accuracy decreased from 0.8338 to 0.8175. Bootstrap-specific balanced weights produced macro-F1 of 0.6147 and minority recall of 0.4073. Global balanced weighting therefore gave the best primary score. The accuracy reduction reflected the intended shift away from majority-class dominance.

The final confirmation experiment exposed an interaction between weighting and leaf size. Table II shows that leaf size five remained within one standard error of the highest macro-F1. Minority recall increased from 0.4061 to 0.5094. Accuracy decreased by 0.0091. This increase in racall with minimum reduction in overall accuracy lead to rather selecting leaf size five for the final model. All-feature and depth-30 alternatives did not improve macro-F1.

TABLE II
FOCUSED CONFIRMATION RESULTS

Candidate	Macro-F1	Minority recall	Accuracy
Leaf 1	0.6162	0.4061	0.8175
Depth 30	0.6151	0.4095	0.8160
Leaf 5	0.6147	0.5094	0.8084
All features	0.6101	0.4149	0.8139

C. Overall Model Comparison

Table III gives the common test results. The forest achieved test macro-F1 of 0.6357. The tree and KNN achieved 0.5899 and 0.4751, respectively. The forest exceeded the tree by 0.0458 macro-F1. MCC increased by 0.0027. Kappa increased by 0.0001. Accuracy decreased from 0.8244 to 0.8197.

TABLE III
CROSS-VALIDATION AND HELD-OUT TEST PERFORMANCE

Measure	KNN	Tree	Forest
CV macro-F1	0.4447 ± 0.0048	0.5709 ± 0.0048	0.6147 ± 0.0058
Test accuracy	0.7857	**0.8244**	0.8197
Test macro-F1	0.4751	0.5899	**0.6357**
Cohen's kappa	0.7197	0.7698	**0.7699**
MCC	0.7204	0.7710	**0.7737**

Figure 3 visualises the overall results. The forest produced the strongest balanced performance rather than the highest number of correct predictions. The primary hypothesis was supported for macro-F1. The reduced accuracy result showed the expected cost of stronger minority sensitivity.

D. Class-Specific Performance

Table IV reports the class-specific forest results. Figure 4 compares recall across all models. Relative to the tree, forest recall increased from 0.1047 to 0.3932 for class 2. Recall increased from 0.3702 to 0.7584 for class 3. Classes 6, 7, and 8 also increased to 0.7116, 0.8649, and 0.9078. The corresponding F1 values improved because recall gains were large enough to offset reduced precision.

Class 5 remained difficult to predict. Forest recall was 0.0671, compared with 0.0493 for the tree and 0.0730 for KNN. The low value shows that balanced weighting did not resolve every minority class. Class 4 recall decreased from

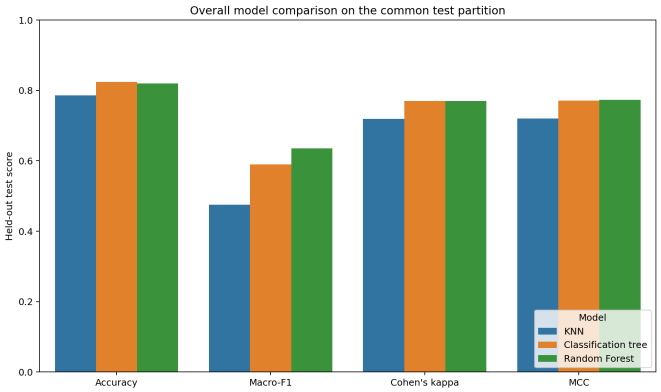


Fig. 3. Overall performance on the common held-out partition.

TABLE IV
CLASS-SPECIFIC HELD-OUT PERFORMANCE OF THE SELECTED FOREST

Class	Precision	Recall	F1	Support	Class	Precision	Recall	F1	Support
0	0.9450	0.8951	0.9193	18,515	5	0.2297	0.0671	0.1038	507
1	0.8858	0.8147	0.8488	2,855	6	0.6511	0.7116	0.6800	4,968
2	0.1524	0.3932	0.2197	468	7	0.6400	0.8649	0.7356	37
3	0.3754	0.7584	0.5022	3,096	8	0.5185	0.9078	0.6600	293
4	0.8669	0.5844	0.6981	8,679	9	0.9979	0.9814	0.9896	12,001

0.7981 for the tree to 0.5844 for the forest. Many class 4 observations were assigned to class 3. The confusion between these two classes indicates overlapping learned regions between those attack categories. Class 0 recall also decreased, while precision increased.

Class 7 requires cautious interpretation. The forest identified 32 of 37 observations. The resulting recall of 0.8649 is high, but the support is too small for a stable estimate. The same caution applies to conclusions about the rarest attacks.

E. Interpretation and Limitations

Figure 5 presents impurity-based importance. Source bytes had the largest value. Mean source packet size, Domain Name System service, recent destination-source counts and source time-to-live followed. The missing-service indicator also ranked highly. Missingness therefore contained predictive information and justified retention as a category.

The empirical design has a couple limitations. First, a single outer test partition does not quantify variation across independent train-test repetitions. Second, KNN tuning used only 60,000 training observations, whereas tree and forest tuning used the full training partition. The common test partition remains the fairest direct comparison. Third, sequential tuning evaluates fewer interactions than a full factorial search. The focused check performed after sequential tuning reduced the effect of this limitation but did not eliminate it. Fourth, class weighting improved recall by accepting more false positives. Operational deployment would require attack-specific costs constraints.

The final forest required 104.5 seconds to fit and 0.18 seconds to predict 51,419 test observations on the available system. The stored forest was less interpretable and larger than a single tree. The predictive gain therefore involved computational and interpretability costs. The main research question was nevertheless answered positively because macro-F1 and several rare-class recalls improved on the common test partition.

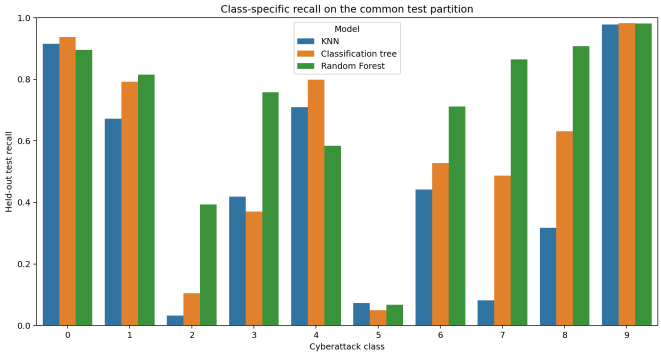


Fig. 4. Class-specific recall on the common held-out partition.

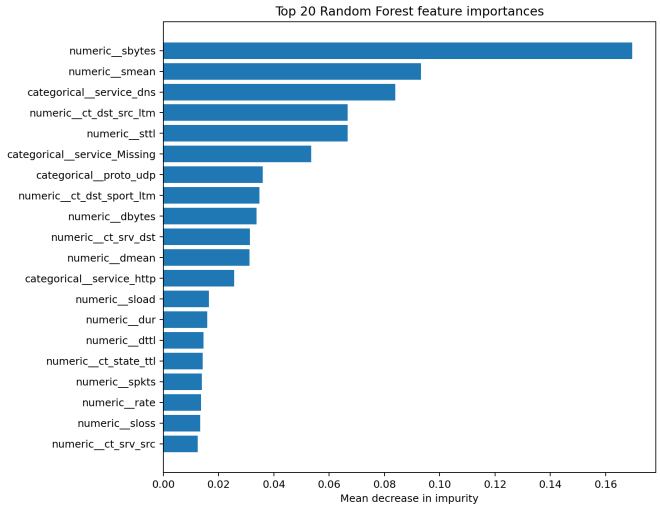


Fig. 5. Twenty largest impurity-based feature importances.

VI. CONCLUSIONS

The study investigated whether a tuned random forest improved multiclass cyberattack prediction relative to k-nearest neighbours and a classification tree. The selected ensemble contained 200 balanced, entropy-based trees. Half of the transformed predictors were considered at each split. Trees had unrestricted depth and a minimum leaf size of five.

Question-driven tuning showed that performance stabilised by 200 trees. Strong feature restriction caused underfitting. Deep trees improved validation performance. Balanced class weighting produced the largest macro-F1 gain.

The selected forest achieved test macro-F1 of 0.6357. The classification tree and k-nearest neighbours achieved 0.5899 and 0.4751. The ensemble improved recall substantially for classes 2, 3, 6, 7, and 8. Accuracy decreased slightly, class 4 recall declined and class 5 remained difficult. The hypothesis was therefore supported for balanced multiclass performance,

but not for every class or every metric. Future work could evaluate repeated outer partitions, attack-specific costs and permutation-based interpretation without revisiting the frozen test result.

APPENDIX A
ACRONYMS

CV	cross-validation
DNS	Domain Name System
KNN	k-nearest neighbours
MCC	Matthews correlation coefficient
RF	random forest
UNSW-NB15	University of New South Wales Network-Based 2015

APPENDIX B
SYMBOLS

Symbol	Definition
B	number of ensemble members
C	number of classes
c	class index
h_b	prediction function of tree b
n	number of training observations
n_c	training observations in class c
p_c	node proportion belonging to class c
w_c	balanced weight for class c
$\boldsymbol{x}$	predictor vector
$\hat{y}$	predicted class

REFERENCES

[1] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[2] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967, doi: 10.1109/TIT.1967.1053964.

[3] S. F. Crone, S. Lessmann, and R. Stahlbock, "The impact of preprocessing on data mining: An evaluation of classifier sensitivity in direct marketing," *European Journal of Operational Research*, vol. 173, no. 3, pp. 781–800, 2006, doi: 10.1016/j.ejor.2005.07.023.

[4] T. G. Dietterich, "Ensemble methods in machine learning," in *Proceedings of the First International Workshop on Multiple Classifier Systems*, 2000, pp. 1–15, doi: 10.1007/3-540-45014-9_1.

[5] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009, doi: 10.1109/TKDE.2008.239.

[6] W.-Y. Loh, "Classification and regression trees," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 1, no. 1, pp. 14–23, 2011, doi: 10.1002/widm.8.

[7] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proceedings of the Military Communications and Information Systems Conference*, 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.

[8] D. Opitz and R. Maclin, "Popular ensemble methods: An empirical study," *Journal of Artificial Intelligence Research*, vol. 11, pp. 169–198, 1999, doi: 10.1613/jair.614.

[9] Z.-X. Qi, H.-Z. Wang, and A.-J. Wang, "Impacts of dirty data on classification and clustering models: An experimental evaluation," *Journal of Computer Science and Technology*, vol. 36, no. 4, pp. 806–821, 2021, doi: 10.1007/s11390-020-0006-6.

[10] O. Rainio, J. Teuho, and R. Klén, "Evaluation metrics and statistical tests for machine learning," *Scientific Reports*, vol. 14, Art. no. 6086, 2024, doi: 10.1038/s41598-024-56706-x.

[11] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009, doi: 10.1016/j.ipm.2009.03.002.